AWS Database Blog

Migrate Microsoft Azure SQL Database to Amazon RDS for SQL Server using Smart Bulk Copy

by Jitendra Kumar and InduTeja Aligeti | on 11 APR 2024 | in Amazon RDS, Intermediate (200), Multicloud, RDS for SQL Server, Technical How-to | Permalink | Comments | Share | Add AWS as a preferred source on search

Resources
Amazon Aurora
Amazon DocumentDB
Amazon DynamoDB
Amazon ElastiCache
Amazon Keyspaces (for Apache Cassandra)
Amazon MemoryDB
Amazon Neptune
Amazon Neptune Analytics
Amazon RDS
Amazon Timestream
AWS Database Migration Service

Amazon Relational Database Service (Amazon RDS) for SQL Server simplifies the process of setting up, operating, and scaling SQL Server deployments in the cloud. By managing time-consuming database administration tasks, such as provisioning, backups, software patching, monitoring, and hardware scaling, Amazon RDS allows you to focus on optimizing database design, capacity planning, and performance tuning.

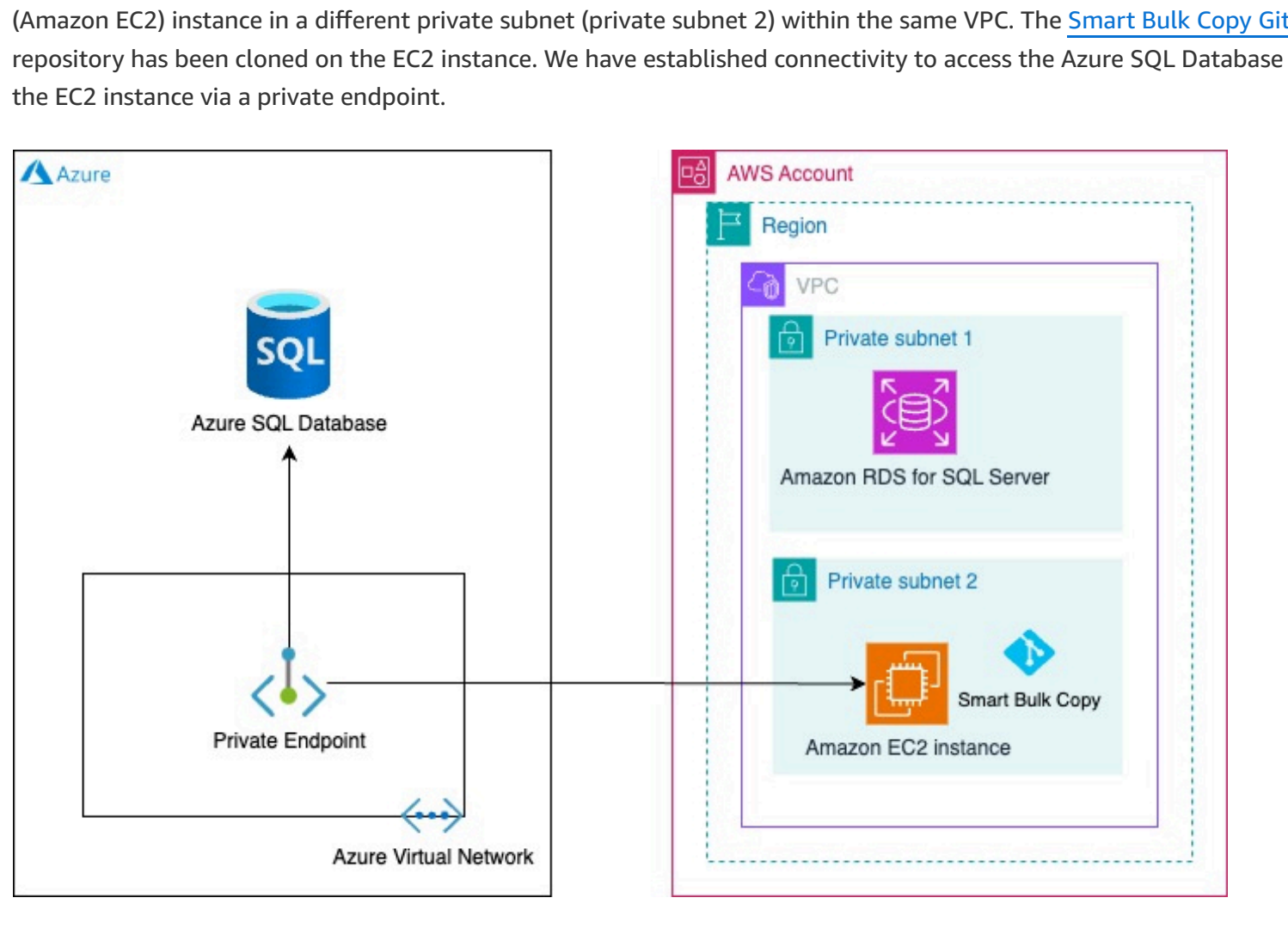
Smart Bulk Copy offers significant benefits for customers seeking to migrate their multi-terabyte Microsoft Azure SQL Database to Amazon RDS for SQL Server. This approach facilitates faster data migration and minimizes downtime, surpassing the effectiveness of traditional tools like bcp and bacpac. Moreover, Smart Bulk Copy accelerates the migration process by partitioning tables for parallel processing. By automating pre-migration and post-migration tasks, it further reduces the potential downtime associated with the migration process.

In this post, we outline the steps to migrate an Azure SQL Database to Amazon RDS for SQL Server using Smart Bulk Copy, a method that offers significantly faster migration compared to the SqlPackage export/import approach. Additionally, we discuss best practices and optimizations to minimize downtime during the migration process.

Smart Bulk Copy is an open-source offline data migration tool developed by Microsoft. It uses the Bulk Copy API with parallel tasks. The source table is partitioned, and each partition is copied in parallel with others, up to a defined maximum limit, to utilize the available network bandwidth and cloud or server resources efficiently. This approach aims to minimize the overall load time for the migration process

Solution overview

In the following architecture diagram, we have created our Azure SQL Database in a private subnet within the Microsoft Azure Cloud. We have an RDS for SQL Server instance in the same AWS Region (where the Azure SQL Database resides) with a Single-AZ deployment (private subnet 1) on AWS. Additionally, we have created an Amazon Elastic Compute Cloud (Amazon EC2) instance in a different private subnet (private subnet 2) within the same VPC. The Smart Bulk Copy GitHub repository has been cloned on the EC2 instance. We have established connectivity to access the Azure SQL Database from the EC2 instance via a private endpoint.



In the following sections, we discuss the pre-migration, migration, and post-migration tasks.

Prerequisites

To implement this solution, complete the following prerequisites:

1. Because the Smart Bulk Copy tool is coded in .Net programming language, make sure you have .NET SDK 3.1.
2. Make sure you have .Net SDK 6.0 in order to use SqlPackage to migrate the database schema.

Use the following code to download and install the .Net SDKs on Amazon EC2:

```
sudo su -

wget https://download.visualstudio.microsoft.com/download/pr/e89c4f00-5cbb-4810-897d-f5300

mkdir -p $HOME/dotnet &&  tar zxf dotnet-sdk-3.1.426-linux-x64.tar.gz  -C $HOME/dotnet

wget https://download.visualstudio.microsoft.com/download/pr/01292c7c-a1ec-4957-90fc-3f6a2

tar zxf dotnet-sdk-6.0.418-linux-x64.tar.gz  -C $HOME/dotnet
```

3. Set up the connectivity between Azure and AWS using a public or private endpoint. Refer to Designing private network connectivity between AWS and Microsoft Azure for more information.
4. Create a SQL Server DB instance with Single-AZ.
5. Create the database on the target server.
6. Make sure the admin account's name for the RDS for SQL Server database is different than the Azure SQL Database.
7. Disable the automated backup for the RDS instance.
8. Make sure that Git is installed on the EC2 (Linux) instance.

Pre-migration activities

You can perform the following pre-migration activities 1 week before the actual migration to reduce the production downtime window.

Create a login and user to migrate the database schema and data

This is required to migrate the database schema and data from Azure SQL Database to Amazon RDS for SQL Server. First, you have to create a login `aws_sbc_user` in the master database, then create a user with the same name and grant necessary permissions to extract the schema definition and read the data from the source database.
Run the following SQL commands against the `master` database:

```
use [master]
go

create login [aws_sbc_user] with password = '<ReplaceWithValidPassword>'
go
create user [aws_sbc_user] for login [aws_sbc_user] with default_schema = [dbo]
go
alter role [loginmanager] add member [aws_sbc_user]
go
```

Run the following SQL commands against your user database (replace <@DBNAME> with the source Azure SQL Database name):

```
use [@DBNAME]
go
create user [aws_sbc_user] for login [aws_sbc_user]
go
alter role [db_ddladmin] add member [aws_sbc_user]
go
alter role [db_datareader] add member [aws_sbc_user]
go

grant view definition to [aws_sbc_user]
go
grant view security definition to [aws_sbc_user]
go
grant select on sys.sql_expression_dependencies to [aws_sbc_user]
go
grant view definition on database::[@DBNAME] to [aws_sbc_user]
go
grant view database performance state to [aws_sbc_user]
go
```

Migrate login, user, and permissions from Azure SQL Database to Amazon RDS

Azure SQL Database is a fully managed platform as a service (PaaS) database engine offering. The login, user, and permissions are migrated by SqlPackage, but the same password will not work in Amazon RDS for SQL Server because the SID is different between Azure and Amazon RDS. We recommend creating the logins with the default password and resetting the password on the target RDS for SQL Server database.

You can use this opportunity to clean up the stale or unused logins, users, and permissions. Follow the least-privilege security best practices.

1. Run the following SQL command against the `master` database to create logins. Do the same for all valid logins. Replace <'ReplaceWithValidPassword'> with any valid default password.

```
use [master]
go

IF NOT EXISTS (select [name] from sys.sql_logins where name =N'dbuser01' and type_desc
BEGIN
     create login [dbuser01] with password = '<ReplaceWithValidPassword>'
END
```

2. Run the SQL command against the **[@DBNAME]** user database:

```
use [@DBNAME]
go

IF NOT EXISTS (select  [name] from sys.sysusers where name =N'dbuser01')
BEGIN
    create user [dbuser01] for login [dbuser01] with default_schema=[dbo]
END

exec sys.sp_addrolemember @rolename=N'db_datareader', @membername=N'dbuser01'

exec sys.sp_addrolemember @rolename=N'db_datawriter', @membername=N'dbuser01'

grant connect to [dbuser01]
```

3. Reset the password:

```
ALTER LOGIN dbuser01 WITH PASSWORD = '<enterStrongPasswordHere>'
```

Migrate the database schema

You can use SqlPackage to migrate the schema from source to target database. Alternatively, you can use SQL Server Management Studio (SSMS) or any other tool to migrate the database schema.
Extract the database schema from the source Azure SQL Database with the following code:

```
mkdir -p $HOME/data
sqlpackage /Action:Extract /TargetFile:"$HOME/data/<database_name>.dacpac" /DiagnosticsFil
```

Publish the database schema to the target RDS for SQL Server database with the following code:

```
cd $HOME/data
sqlpackage /Action:Publish /SourceFile:"$HOME/data/<database-name>.dacpac" /DiagnosticsFil
```

You must replace the placeholders accordingly:

- **<database-name>** – Name of the database
- **<azusre-sql-server-name>** – Name of the Azure SQL Server
- **<source-user-name>** – Source user name
- **<source-user-password>** – Source user password
- **<rds-sql-server-name>** – RDS for SQL Server endpoint
- **<target-user-name>** – Target user name
- **<target-user-password>** – Target user password

Validate the schema has been migrated successfully.

Launch an EC2 instance

You have to launch an EC2 (Linux) instance in order to install, configure, and run the Smart Bulk Copy tool to migrate the data.

Alternatively, you can use a Docker image to run Smart Bulk Copy via Docker.

Drop objects

You should drop the following objects before running Smart Bulk Copy to migrate the data. We recommend taking a backup of these objects before dropping them so that you can create them after migrating the data.

- Views
- Foreign keys
- Secondary indexes
- Triggers
- Primary keys and unique constraints from big tables

Dropping the secondary indexes, primary keys, and unique constraints will expedite the data load.

Migration activities

In this phase, you migrate the data from Azure SQL Database to Amazon RDS for SQL Server using Smart Bulk Copy. Make sure that you set the source database to READ ONLY mode before running Smart Bulk Copy and get the binary checksum from the source. When the data load is complete, you can get the binary checksum from the target and compare the checksum values between source and target to validate the data.

Clone the Smart Bulk Copy repository

Connect to the EC2 instance either by using Session Manager, a capability of AWS Systems Manager, or by using an SSH client. Run the following commands to clone the Smart Bulk Copy GitHub repository:

```
sudo su  -

git clone https://github.com/Azure-Samples/smartbulkcopy.git
cd smartbulkcopy
ls -lrt
```

Build Smart Bulk Copy

Run the following command to build Smart Bulk Copy:

```
cd smartbulkcopy/client

dotnet build
```

Set up smartbuilcopy.config

To configure the `smartbulkcopy.config` file, complete the following steps:

1. Go to the `smartbulkcopy` directory:
   `cd ~/smarbulkcopy/client`
2. Copy the template file from the config directory to `~/smartbulkcopy/client`
```
cp configs/smartbulkcopy.config.template.json .

           mv smartbulkcopy.config.template.json smartbulkcopy.config
```
3. Update the `smartbulkcopy.config` file.

Smart Bulk Copy is highly configurable. For more details about each parameter, refer to Configuration Options. The following is our sample smartbulkcopy.config file:

```
{
    "source":{
        "connection-string": "Server=<azure-sql-server-name>,1433;Initial Catalog=<datab
    },
    "destination":{
        "connection-string": "Server=<rds-sql-server-name>,1433;Initial Catalog=<databas
    },
    "tables": {
        "include": ["*"],
        "exclude": ["dbo.MEM_OPT_TABLE1 ","dbo.MEM_OPT_TABLE2"]
    },
    "options": {
        "compatibility-mode": false,
        "command-timeout": 1800,
        "tasks": 8,
        "logical-partitions": "auto",
        "batch-size": 100000,
        "truncate-tables": true,
```

Run Smart Bulk Copy

Run the following command to trigger Smart Bulk Copy to migrate the data:

```
cd ~/smarbulkcopy/client
dotnet run
```

Monitor the output of the `dotnet run` command.

Smart Bulk Copy first reads the `smartbulkcopy.config` configuration file and tests the connection for source and target databases. It then analyzes all the tables to identifies whether it should use parallel load for a table or not.

You can observe the behavior of the parallel load, as shown in the following screenshot.

For `[dbo].[test_table_new]`, although the parallel load option is available, Smart Bulk Copy doesn't use parallel threads because the table is small (less than 1 GB).

For `[dbo].[xxxx_history]` and `[dbo].[xxxx_session]`, you can see the message "*Source and destination are not partitioned and both are heaps.*" Smart Bulk Copy uses parallel threads and splits tables into five and three logical partitions, respectively, because these tables are large (more than 1 GB).

For `[dbo].[xxxx]` tables, you can see the message "*Source is not partitioned but destination is an heap.*" Smart Bulk Copy uses parallel threads because the table is large (more than 1 GB).
Smart Bulk Copy prints a message when the partition of a table gets copied.

Smart Bulk Copy verifies the row count between the source and target tables after all the tables are loaded. It also prints the time the taken to complete the load (*Smart Bulk Copy completed in 357.09 secs*).

```
2024/02/09 18:04:34.741|INFO> Table [dbo].[test_table_new] has 0 rows both in source and d

2024/02/09 18:04:34.741|INFO> Table [dbo].[uxxx_history] has 10485760 rows both in source

2024/02/09 18:04:34.744|INFO> Table [dbo].[uxxx_session] has 3401280 rows both in source a

2024/02/09 18:04:34.744|INFO> Table [dbo].[uxxxs] has 1966080 rows both in source and dest

2024/02/09 18:04:34.749|INFO> Table [Sales].[Customer] has 19820 rows both in source and d

2024/02/09 18:04:34.828|INFO> All tables copied correctly.

2024/02/09 18:04:34.828|INFO> Smart Bulk Copy completed in 357.09 secs.
```

Post-migration activities

Before we started the data migration, we dropped the constraints or referenced objects like primary keys, indexes, reference keys, and views. Therefore, after you have migrated the data to Amazon RDS for SQL Server, you must complete the following steps on the target:

1. Create primary keys, unique constraints, and indexes.
2. Create foreign keys.
3. Create views.
4. Create triggers.
5. Migrate memory-optimized tables using the bcp utility.
6. Migrate sequences.

In addition to the preceding tasks, Azure SQL Database includes default settings for Transparent Data Encryption (TDE) for Azure SQL Database, auditing for Azure SQL Database, and diagnostic settings in Azure Monitor. Conversely, with Amazon RDS for SQL Server, these functionalities need to be manually enabled. Refer to Turning on TDE for RDS for SQL Server, Auditing Amazon RDS for SQL Server DB instances, and Monitoring DB load with Performance Insights on Amazon RDS for more details.
Using Smart Bulk Copy, we were able to migrate approximately 1 TB of data in around 1 hour, 30 minutes, excluding the post-migration activities. The source Azure SQL Database was running on Business Critical: Gen5, 32vCores, the RDS for SQL Server instance size was db.r6i.8xlarge (32 vCPU and 256 GiB memory with 32000 provisioned IOPS SSD storage), and the Smart Bulk Copy EC2 instance size was r6i.8xlarge (32 vCPU and 256 GB memory) with 32 parallel tasks. We recommend testing this solution thoroughly because the performance may differ based on the network bandwidth, number of large tables, and maximum size of the largest table.

Limitations

This solution has the following limitations:

- Smart Bulk Copy doesn't support memory-optimized tables. You can use the bcp utility to migrate these tables.
- Sequences must be migrated separately.
- Smart Bulk Copy is an offline data migration tool. Therefore, the database schema needs to migrated separately.
- Foreign keys, secondary indexes, and other objects need to be dropped separately. Refer to Smart Bulk Copy FAQs for more information.
- Smart Bulk Copy doesn't support incremental (change data capture) load.
- Data validation only compares the row count between the source and target table.

Advantages

This solution offers the following benefits:

- Smart Bulk Copy is an open source tool developed and managed by Microsoft.
- You can use this solution to perform failback (rollback) testing. You have to modify the `smartbulkcopy.config` file and modify the source and target connection string to migrate data from Amazon RDS for SQL Server to Azure SQL Database.
- Smart Bulk Copy provides parallel loading for large tables by splitting into partitions. For more information, refer to Observed Performances.

Best practices

Consider the following best practices:

- The source Azure SQL database must be set to *READ ONLY* before you run Smart Bulk Copy to avoid data discrepancies between source and target.
- Test this solution for a medium size table (10–20 GB) and monitor the time taken to load the data. After you have benchmarked the time, run it for all tables.
- We recommend allocating the appropriate size for DATA and LOG files based on your source database size in Amazon RDS for SQL Server to avoid frequent *auto-grow* operation overhead while running Smart Bulk Copy.
- Increase the *TEMPDB* size as per your requirement to avoid *auto-grow* operation overhead while creating a large index during post-migration activities.

Clean up

To avoid future charges, remove all the resources you created while testing the Smart Bulk Copy solution by completing the following steps:

1. On the Amazon RDS console, choose Databases in the navigation pane.
2. Select the databases you set up and on the Actions menu, choose Delete.
3. Enter *delete me* to confirm deletion.
4. For more information about deleting an RDS instance, refer to Deleting a DB instance.
5. On the Amazon EC2 console, find the EC2 instance that you used to run Smart Bulk Copy and delete the instance.

Conclusion

In this post, we explored how you can migrate Azure SQL databases to Amazon RDS for SQL Server using Smart Bulk Copy. With a successful migration to Amazon RDS for SQL Server, you can concentrate on aspects that are essential to your organization's business, such as optimizing database design, capacity planning, and performance tuning.
If you have any questions or suggestions about this post, leave them in the comment section.

About the Authors

Jitendra Kumar is a Lead Database Migration Consultant at AWS with over 20 years of working experience in enterprise databases like Db2, SAP ASE, and SQL Server. He focuses on helping customers migrate and modernize their workloads to AWS.

InduTeja Aligeti is a Senior Lead Database Consultant at AWS. She has over 18 years of experience working with Microsoft technologies with a specialization in SQL Server. She focuses on helping customers build highly available, cost-effective database solutions and migrate their large-scale SQL Server databases to AWS.

Like Share

Comments

Log in to comment

Learn

What Is AWS?
What Is Cloud Computing?
What Is Agentic AI?
Cloud Computing Concepts Hub
AWS Cloud Security
What's New
Blogs
Press Releases

Resources

Getting Started
Training
AWS Trust Center
AWS Solutions Library
Architecture Center
Product and Technical FAQs
Analyst Reports
AWS Partners

Developers

Builder Center
SDKs & Tools
.NET on AWS
Python on AWS
Java on AWS
PHP on AWS
JavaScript on AWS

Help

Contact Us
File a Support Ticket
AWS re:Post
Knowledge Center
AWS Support Overview
Get Expert Help
AWS Accessibility
Legal

Back to top ↑

Amazon is an Equal Opportunity Employer: Minority / Women / Disability / Veteran / Gender Identity / Sexual Orientation / Age.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.